

```
[apps/api/main.py] from future import annotations import json import
mimetypes import os import secrets from http import HTTPStatus from
http.server import BaseHTTPRequestHandler, ThreadingHTTPServer from
pathlib import Path from urllib.parse import parse_qs, urlparse from
analytics import build_alerts, dashboard_snapshot,
generate_recommendation, predict_section, risk_level, risk_score from
repository import ProjectRepository ROOT = Path(file).resolve().parents[2]
WEB_ROOT = ROOT / "apps" / "web" DATA_PATH = ROOT / "apps" / "api" /
"data" / "demo_dataset.json" CONTRACT_PATH = ROOT / "packages" /
"shared- contracts" / "api_contract.json" RULEBOOK_PATH = ROOT /
"packages" / "shared- contracts" / "support_rulebook.json" PORT =
int(os.environ.get("APP_PORT", "8010")) SESSIONS: dict[str, dict[str, str]] =
{} REPOSITORY = ProjectRepository(DATA_PATH) def
json_response(handler: BaseHTTPRequestHandler, status: int, payload: dict)
-> None: body = json.dumps(payload, ensure_ascii= False).encode("utf-8")
handler.send_response(status) handler.send_header("Content-Type",
"application/json; charset=utf-8") handler.send_header("Content-Length",
str(len(body))) handler.end_headers() handler.wfile.write(body) class
TunnelAssistantHandler(BaseHTTPRequestHandler) : server_version =
"TunnelAssistant/1.0" def do_GET(self) -> None: # noqa: N802 parsed =
urlparse(self.path) if parsed.path.startswith("/api/"):
self.handle_api_get(parsed) return self.serve_static(parsed.path) def
do_POST(self) -> None: # noqa: N802 parsed = urlparse(self.path) if not
parsed.path.startswith("/api/"): json_response(self,
HTTPStatus.NOT_FOUND, {"ok": False, "message": "unsupported"}) return
self.handle_api_post(parsed) def log_message(self, fmt: str, *args) -> None:
```

noqa: A003

```
return
def parse_json_body(self) -> dict:
    length = int(self.headers.get("Content-
Length", "0")) raw = self.rfile.read(length) if length else b"{}" if not raw:
return {} return json.loads(raw.decode("utf-8")) def current_user(self) ->
dict | None: auth = self.headers.get("Authorization", "") if not
auth.startswith("Bearer "): return None token = auth.replace("Bearer ", "", 1)
return SESSIONS.get(token) def require_user(self) -> dict | None: user =
self.current_user() if user is None: json_response(self,
HTTPStatus.UNAUTHORIZED, {"ok": False, "message": "未登录或令牌失
效。"}) return None return user def serve_static(self, raw_path: str) -> None:
if raw_path in {"/", ""}: target = WEB_ROOT / "index.html" elif raw_path ==
"/assets/styles.css": target = WEB_ROOT / "styles.css" elif raw_path ==
```

```

"/assets/app.js": target = WEB_ROOT / "app.js" else: target = WEB_ROOT /
raw_path.lstrip("/") if not target.exists() or not target.is_file():
self.send_error(HTTPStatus.NOT_FOUND) return mime, _ =
mimetypes.guess_type(str(target)) content = target.read_bytes()
self.send_response(HTTPStatus.OK) self.send_header("Content-Type", f"
{mime or 'application/octet-stream'}; charset=utf- 8")
self.send_header("Content-Length", str(len(content))) self.end_headers()
self.wfile.write(content) def handle_api_get(self, parsed) -> None: user =
None if parsed.path not in {"/api/health", "/api/ contracts"}: user =
self.require_user() if user is None: return if parsed.path == "/api/health":
json_response(self, HTTPStatus.OK, {"ok": True, "service": "tunnel-assistant",
"port": PORT}) return if parsed.path == "/api/session": json_response(self,
HTTPStatus.OK, {"ok": True, "user": user}) return if parsed.path ==
"/api/dashboard": dataset = REPOSITORY.load() payload =
dashboard_snapshot(dataset) payload["auditTrail"] = dataset["auditTrail"]
[-5:] json_response(self, HTTPStatus.OK, {"ok": True, "data": payload}) return
if parsed.path == "/api/sections": params = parse_qs(parsed.query)
keyword = params.get("keyword", [""])[0] risk = params.get("risk", [""])[0]
sections = REPOSITORY.search_sections(key word=keyword, risk=risk)
payload = [] for section in sections: payload.append( { **section,
"riskScore": risk_score(section), "riskLevel": risk_level(risk_score(section)), } )
json_response(self, HTTPStatus.OK, {"ok": True, "items": payload}) return if
parsed.path.startswith("/api/sections/"): section_id = parsed.path.rsplit("/",
1)[-1] section = REPOSITORY.get_section(section_id) if section is None:
json_response(self, HTTPStatus.NOT_FOUND, {"ok": False, "message": "区段
不存在。"}) return json_response( self, HTTPStatus.OK, { "ok": True, "item": {
**section, "riskScore": risk_score(section), "riskLevel":
risk_level(risk_score(section)), }, }, ) return if parsed.path == "/api/alerts":
dataset = REPOSITORY.load() json_response(self, HTTPStatus.OK, {"ok":
True, "items": build_alerts(dataset["sections"])} return if parsed.path ==
"/api/predictions": json_response(self, HTTPStatus.OK, {"ok": True, "items":
REPOSITORY.recent_predictions( )}) return if parsed.path ==
"/api/recommendations": json_response(self, HTTPStatus.OK, {"ok": True,
"items": REPOSITORY.recent_recommendati ons()}) return if parsed.path ==
"/api/contracts": payload = { "api": json.loads(CONTRACT_PATH.read
_text(encoding="utf-8")) if CONTRACT_PATH.exists() else {}, "rulebook":
json.loads(RULEBOOK_PATH.read_text(encoding="utf- 8")) if
RULEBOOK_PATH.exists() else {}, } json_response(self, HTTPStatus.OK, {"ok":
True, "data": payload}) return json_response(self, HTTPStatus.NOT_FOUND,
{"ok": False, "message": "接口不存在。"}) def handle_api_post(self, parsed) -
> None: if parsed.path == "/api/auth/login": body = self.parse_json_body()
user = REPOSITORY.get_user(body.get("use rname", "")) if user is None or
user["password"] != body.get("password", ""): json_response(self,
HTTPStatus.UNAUTHORIZED, {"ok": False, "message": "用户名或密码错
误。"}) return token = secrets.token_hex(16) session_user = { "username":
user["username"], "name": user["name"], "role": user["role"], "title":
user["title"], "permissions": user["permissions"], } SESSIONS[token] =

```

```

session_user json_response(self, HTTPStatus.OK, {"ok": True, "token": token,
"user": session_user}) return user = self.require_user() if user is None: return
body = self.parse_json_body() if parsed.path == "/api/auth/logout": auth =
self.headers.get("Authorization", "") token = auth.replace("Bearer ", "", 1) if
token: SESSIONS.pop(token, None) json_response(self, HTTPStatus.OK,
{"ok": True}) return if parsed.path == "/api/predictions": section =
REPOSITORY.get_section(body.get("sectionId", "")) if section is None:
json_response(self, HTTPStatus.BAD_REQUEST, {"ok": False, "message": "未
找到指定区段。"}) return dataset = REPOSITORY.load() scenario = {
"scenarioName": body.get("scenarioName", "掌子面推进情景"),
"generatedAt": body.get("generatedAt", "2026-05-15T11:00:00"),
"excavationIntensity": body.get("excavationIntensity", 1.0), "rainfallFactor":
body.get("rainfallFactor", 0.5), "safetyFactor": body.get("safetyFactor", 1.15),
} prediction = predict_section(section, scenario,
dataset["supportTemplates"]) REPOSITORY.append_prediction(prediction)
json_response(self, HTTPStatus.OK, {"ok": True, "item": prediction}) return if
parsed.path == "/api/recommendations": section =
REPOSITORY.get_section(body.get("sectionId", "")) if section is None:
json_response(self, HTTPStatus.BAD_REQUEST, {"ok": False, "message": "未
找到指定区段。"}) return dataset = REPOSITORY.load() scenario = {
"scenarioName": body.get("scenarioName", "支护复核情景"), "generatedAt":
body.get("generatedAt", "2026-05-15T11:10:00"), "excavationIntensity":
body.get("excavationIntensity", 1.0), "rainfallFactor":
body.get("rainfallFactor", 0.5), "safetyFactor": body.get("safetyFactor", 1.15),
} recommendation = generate_recommendation(section, scenario,
dataset["supportTemplates"])
REPOSITORY.append_recommendation(recommendation)
json_response(self, HTTPStatus.OK, {"ok": True, "item": recommendation})
return json_response(self, HTTPStatus.NOT_FOUND, {"ok": False, "message":
"不支持的提交。"}) def run() -> None: REPOSITORY.ensure_seeded() httpd =
ThreadingHTTPServer(("127.0.0.1", PORT), TunnelAssistantHandler)
print(f"Tunnel assistant running on http:// 127.0.0.1:{PORT}")
httpd.serve_forever() if name == "main": run() [apps/web/app.js] const
state = { token: localStorage.getItem("tunnel_token") || "", user: null,
dashboard: null, sections: [], selectedSection: null, predictions: [],
recommendations: [], alerts: [], contracts: null, filters: { keyword: "", risk: "",
}, scenario: { sectionId: "SEC-005", scenarioName: "甯歌口鉏×繙鄧呖檻",
excavationIntensity: 1.0, rainfallFactor: 0.5, safetyFactor: 1.15, }, }; const
loginPanel = document.querySelector("#login- panel"); const appShell =
document.querySelector("#app- shell"); const contentArea =
document.querySelector("#content -area"); const userPill =
document.querySelector("#user- pill"); const loginMessage =
document.querySelector("#login- message"); const loginForm =
document.querySelector("#login- form"); const logoutBtn =
document.querySelector("#logout- btn"); const navLinks =
[...document.querySelectorAll(".sid enav a")]; function route() { return
window.location.hash.replace("#", "") || "dashboard"; } function

```

```
setActiveNav() { const current = route(); navLinks.forEach((link)=> {  
    link.classList.toggle("active", link.dataset.route === current); }); } async  
function api(path, options = {}) { const headers = new  
Headers(options.headers || {} ); headers.set("Content-Type",  
"application/json"); if (state.token) { headers.set("Authorization", Bearer  
`${state.token}`); } const response = await fetch(path, { ...options, headers, });  
const data = await response.json(); if (response.status === 401) {  
resetSession(); } if (!response.ok || !data.ok) { throw new Error(data.message  
|| "璇锋涰湠弊触"); } return data; } function resetSession() { state.token = "";  
state.user = null; localStorage.removeItem("tunnel_token");  
loginPanel.classList.remove("hidden"); appShell.classList.add("hidden"); }  
function riskClass(level) { return (level || "").toLowerCase(); } function  
riskLabel(level) { const map = { LOW: "浣庡▯闂?, MEDIUM: "涓▯□闕?, HIGH:  
"楂▯閫▯闕?, CRITICAL: "涓寸倝棕摩櫟", }; return map[level] || level; } function  
formatNumber(value, digits = 2) { return Number(value || 0).toFixed(digits);  
} function metricRow(label, value, unit = "") { return <div class="metric-row">  
<span>${label}</span><strong>${value}${unit}</strong></div>; } function  
riskPill(level, score) { return <span class="risk-pill ${riskClass(level)}  
>${riskLabel(level)}${score ? `路` : ""}${formatNumber(score)} : ""></span>; }  
function buildPolyline(values, width, height, color) { const padding = 18;  
const max = Math.max(...values, 1); const min = Math.min(...values, 0);  
const range = Math.max(max - min, 1); const step = (width - padding * 2) /  
Math.max(values.length - 1, 1); const points = values.map((value, index)  
=> { const x = padding + step * index; const y = height - padding - ((value  
- min) / range) * (height - padding * 2); return `${x},${y}; }) .join(" "); return  
<polyline fill="none" stroke="${color}" stroke-width="3" points="${points}"  
></polyline>; } function renderPredictionChart(prediction) { if (!prediction) { return  
<div class="empty-state"> 灑氣酒鑒熳垚棗勭緇綉澶滀鍔犻偤钼悘閮闃夊嫆鎷嶅煞零舵璁哥矖纈纈  
榪€?/></div>; } const settlement = prediction.predictedSettlement; const  
convergence = prediction.predictedConvergen e; return <div class="chart-  
shell"> <svg viewBox="0 0 720 220" role="img" aria-labe l="鏍忎紨棗勭緇綉澶滀鍔加  
鐢�?"> <rect x="0" y="0" width="720" height= "220" fill="transparent"></rect>  
<line x1="18" y1="188" x2="702" y2= "188" stroke="rgba(255,255,255,0.18)" />  
<line x1="18" y1="18" x2="18" y2= "188" stroke="rgba(255,255,255,0.18)" />  
${buildPolyline(settlement, 720, 220, "#ff8a3d")}  
${buildPolyline(convergence, 720, 220, "#58d0db")} </svg> <div class="chart-  
legend"> <span class="legend-settlement"> 鎵嬬画《娌寔櫧棗勭緇綉澶滀鍔加</span> <span  
class="legend-convergence"> 鍛ㄥ厐鍏斿姩杞昏棗勭緇綉澶滀鍔加</span> </div> </div>; } function  
statusTable(rows) { return <table class="section-table"> <thead> <tr> <th>鎷  
烘▯</th> <th>鏍忎紨纋堝墜崙</th> <th>棕摩櫟</th> <th>娌寔櫧 mm</th> <th>鍔动杞昏  
mm</th> <th>鎵�辨灑瘦旛妲 kn</th> <th>鍔ユ▯闕?mm</th> </tr> </thead> <tbody>  
${rows.map( (row) => row.zone </td> < td>{row.rockGrade}  
riskPill(row.riskLevel, row.riskScore) </td> < td>  
{formatNumber(row.settlement)</td> formatNumber(row.convergence)  
</td> < td>{formatNumber(row.archStress)</td>  
${formatNumber(row.delta)}, )}.join("")}} </tbody> </table>; } function  
renderDashboard() { const dashboard = state.dashboard; if (!dashboard) {
```


澶動疆鎖悞ず

```
    ${latestPrediction.notes.map((note) => `
    • ${note}

    `).join("")}

`: "" } `; } function renderDesigns() { const latestRecommendation =
state.recommendations[0]; return `
```

鎬𠂔娟澶嘈姆

灏嗽𠂔嫻孀栢鏊滅洵鋈𠂔槩灏勸垠鎧峰雌娣峰嶠 鎧增帑拏ㄣ€俐馭鍬嗽暱拏ㄣ€侷 僂
嫻鏊墮梘豐澶穿極動睭鑾樺舰閱氛紆褰(-)垚鑾𠂔愰鎧扮琬 鎬𠂔娟澶嘈姆鎰他𠂔
鎰?/p>

鎧轰鎧呼疆浜旂駭鎬𠂔娟璫勸圻娣?/span> 杈嶳語錫𠂔垠鎬𠂔穿寤鸿𠂔鍊煎𠂔
姣?/span>
鎧烘𠂔 𠂔 鄧呖檻錫嶇𠂔 \${state.scenario.scenarioNar} 鎶板婢绯绘暄 閩確
洎绯绘暄

淪攵攴錫樺嘶绯绘暄
鑾燭垚寤鸿𠂔

```
${ latestRecommendation ? `
${latestRecommendation.summary}
```

鍬�枅鑾俗暄

```
${metricRow("鎧鋒販鋈氫害",
latestRecommendation.materialPlan.shotcreteThickness Cm, " cm"))}
${metricRow("閱氣漚閭垮害",
latestRecommendation.materialPlan.boltLengthM, " m"))}
${metricRow("閱氣漚閭磋寇",
latestRecommendation.materialPlan.boltSpacing)} ${metricRow("鎧辨灑
閭磋寇", latestRecommendation.materialPlan.steelArchSpacing)}
${metricRow("極動睭鑾樺舰閱?,
latestRecommendation.materialPlan.reservedDeformatio nMm, "
mm"))}
```

澶嘈姆漢規彥

```
${metricRow("鋈增枒娣峰帑拏?,
latestRecommendation.comparison.initialShotcreteThic knessCm, "
cm"))} ${metricRow("寤鸿𠂔鎧鋒販鋈氫害",
latestRecommendation.comparison.suggestedShotcreteTh icknessCm, "
cm"))} ${metricRow("鋈燭馭鍬嗽暱拏?,
```

```

latestRecommendation.comparison.initialBoltLengthM, " m"))
    ${metricRow("寤鸿閩氣漣閩垮害",
latestRecommendation.comparison.suggestedBoltLengthM , " m"))
    ${metricRow("鋤閩鋤確閩閩+塔",
formatNumber(latestRecommendation.comparison.riskScoreBefore))}
    ${metricRow("鋤閩鋤錫摩閩浼板塔",
formatNumber(latestRecommendation.comparison.riskScoreAfterMeasure))}

```

璫捐渚濇崕

```

    ${latestRecommendation.decisionBasis.map((item) => `
    • ${item}
    `).join("")}

```

錄u鋤閩鋤

```

    ${latestRecommendation.measures.map((item) => `
    • ${item}
    `).join("")}

```

```

`:`
灝氣瀝鑿燭堊鋤閩娟寤鸿閩鋤?/div>` }
`; } function renderAlerts() { return `

```

樞勳澶動疆

鐳嫻E鑿憂潯浹躲€ 伙矗浹浹汉鍛出幫鎢哄姘浣滢啓鎬�塹鐳困絳涓蔣閩樼勳閩鍋
滅睽鎢× " 鐳奸嗽鋤?/p>

鐳閩姘鑿堊閩閩+塔鍊椒帛 涓�敵鐳p閩鐳稿馭浜閩悅涓€ 濂橘脍变典富鐳版崕
 \${state.alerts .map((alert) => `

```

    ${alert.sectionName}

```

```

    ${riskPill(alert.riskLevel, alert.riskScore)}

```

```

    璫〰發鐳'欢鑄?/strong> ${alert.trigger}

```

```

    澶動疆鐳九絛鑄?/strong> ${alert.action}

```

```

    璐d換浜猴綫 ${alert.owner}

```

```

    ` , ) .join("")}
    `; } function renderReports() { const latestPrediction =
    state.predictions[0]; const latestRecommendation =
    state.recommendations[0]; const exampleCommand = `python - X utf8

```


浜や親鰐°C母

鏈€緇續互 pipeline guard 鏄ゆ罈涓哄悗 姝ら〉 璁や簬鎶′竴鍒楅檮娉ㄣ

鍛戒护紺毒綹

鉤、彗、涇、序、錄、權、博、瑕?/h3> \${ state.contracts ? `

錮工 龔妮吡崙

鍬□媼璫勸挾綈?/h3>

```
{state.contracts.rulebook.levels. map((item) =>
```

- `{item.level}` 縮短細鏢鋒販
`{item.shotcreteThickness}` cm 鏢峴啟鏢?
`{item.boltLength}` m 鏢岫鏢鏢?
`{item.steelArchSpacing}`
`).join("")}`

```
`:`
```

```
鈕 彳 彝濂戔害灝氣酒鏢抗澆鉅?/div>` }
```

```
`; } const renderers = { dashboard: renderDashboard,  
sections: renderSections, analysis: renderAnalysis, designs:  
renderDesigns, alarms: renderAlerts, reports: renderReports,  
}; function bindSectionEvents() {  
document.querySelector("#filter-btn")?  
.addEventListener("click", async () => { state.filters.keyword  
= document.querySelector("#keyword- input").value.trim();  
state.filters.risk = document.querySelector("#risk-  
select").value; await loadSections(); render(); });  
document.querySelectorAll(".section- row").forEach((row)  
=> { row.addEventListener("click", async () => { const  
sectionId = row.dataset.sectionId; await  
selectSection(sectionId); window.location.hash = "sections";  
}); }); } function bindAnalysisEvents() {  
document.querySelector("#scenario-section")?  
.addEventListener("change", (event) => {  
state.scenario.sectionId = event.target.value; });  
document.querySelector("#scenario-name")?  
.addEventListener("input", (event) => {  
state.scenario.scenarioName = event.target.value; });  
document.querySelector("#scenario-excavation")?  
.addEventListener("input", (event) => {  
state.scenario.excavationIntensity =  
Number(event.target.value); });  
document.querySelector("#scenario-rainfall")?  
.addEventListener("input", (event) => {  
state.scenario.rainfallFactor = Number(event.target.value);  
}); document.querySelector("#scenario-safety")?  
.addEventListener("input", (event) => {
```

```

state.scenario.safetyFactor = Number(event.target.value); });
document.querySelector("#predict-btn"?
.addEventListener("click", async () => { await
createPrediction(); render(); });
document.querySelector("#load-critical-btn"?
.addEventListener("click", () => { state.scenario = { sectionId:
"SEC-008", scenarioName: "娣卞堪杞□博鑽撮洎錄板婢",
excavationIntensity: 1.35, rainfallFactor: 1.1, safetyFactor:
1.05, }; render(); }); } function bindDesignEvents() {
document.querySelector("#recommend-section"?
.addEventListener("change", (event) => {
state.scenario.sectionId = event.target.value; });
document.querySelector("#recommend-name"?
.addEventListener("input", (event) => {
state.scenario.scenarioName = event.target.value; });
document.querySelector("#recommend-excavation"?
.addEventListener("input", (event) => {
state.scenario.excavationIntensity =
Number(event.target.value); });
document.querySelector("#recommend-rainfall"?
.addEventListener("input", (event) => {
state.scenario.rainfallFactor = Number(event.target.value);
}); document.querySelector("#recommend-safety"?
.addEventListener("input", (event) => {
state.scenario.safetyFactor = Number(event.target.value); });
document.querySelector("#recommend-btn"?
.addEventListener("click", async () => { await
createRecommendation(); render(); }); } function
bindEventsByRoute() { const current = route(); if (current
=== "sections") { bindSectionEvents(); } if (current ===
"analysis") { bindAnalysisEvents(); } if (current ===
"designs") { bindDesignEvents(); } } function render() {
setActiveNav(); const current = route(); const renderer =
renderers[current] || renderDashboard;
contentArea.innerHTML = renderer(); bindEventsByRoute(); }
async function selectSection(sectionId) { const { item } =
await api(`/api/sections/ ${sectionId}`); state.selectedSection
= item; state.scenario.sectionId = item.id; render(); } async
function loadDashboard() { const { data } = await
api("/api/dashboard"); state.dashboard = data; } async

```

```

function loadSections() { const query = new
URLSearchParams(); if (state.filters.keyword) {
query.set("keyword", state.filters.keyword); } if
(state.filters.risk) { query.set("risk", state.filters.risk); } const
{ items } = await api(`/api/ sections${query.toString() ? `?`
`${query.toString()}` : ""}`); state.sections = items; if
(!state.selectedSection && items.length) {
state.selectedSection = items[0]; state.scenario.sectionId =
items[0].id; } } async function loadPredictions() { const {
items } = await api("/api/predictions"); state.predictions =
items; } async function loadRecommendations() { const {
items } = await api("/api/ recommendations");
state.recommendations = items; } async function
loadAlerts() { const { items } = await api("/api/alerts");
state.alerts = items; } async function loadContracts() { const
{ data } = await api("/api/contracts"); state.contracts = data;
} async function createPrediction() { const { item } = await
api("/api/predictions", { method: "POST", body:
JSON.stringify(state.scenario), }); state.predictions = [item,
...state.predictions] .slice(0, 8); } async function
createRecommendation() { const { item } = await
api("/api/recommendations", { method: "POST", body:
JSON.stringify(state.scenario), }); state.recommendations =
[item, ...state.recommendations].slice(0, 8); } async function
bootstrapApp() { loginPanel.classList.add("hidden");
appShell.classList.remove("hidden"); userPill.textContent =
`${state.user.name} / ${state.user.title}`; await
Promise.all([loadDashboard(), loadSections(),
loadPredictions(), loadRecommendations(), loadAlerts(),
loadContracts()]); render(); }
loginForm.addEventListener("submit", async (event) => {
event.preventDefault(); loginMessage.textContent = "姝 e 濠
鏢 嶧 嶧 嶧 嶧 嶧..."; const formData = new FormData(loginForm);
try { const payload = Object.fromEntries(formData.entrie
s()); const { token, user } = await api("/api/auth/ login", {
method: "POST", body: JSON.stringify(payload), });
state.token = token; state.user = user;
localStorage.setItem("tunnel_token", token);
loginMessage.textContent = ""; await bootstrapApp(); }
catch (error) { loginMessage.textContent = error.message; }

```

```
}); logoutBtn.addEventListener("click", async () => { if
(state.token) { try { await api("/api/auth/logout", { method:
"POST", body: "{}" }); } catch (error) { console.warn(error); } }
resetSession(); }); window.addEventListener("hashchange",
async () => { const current = route(); if (current ===
"sections" && state.selectedSection) { await
selectSection(state.selectedSection.id); return; } render(); });
async function resumeSession() { if (!state.token) {
resetSession(); return; } try { const { user } = await
api("/api/session"); state.user = user; await bootstrapApp();
} catch (error) { resetSession(); } } resumeSession();
[apps/web/index.html]
```